

Sección 4 — Juegos

Documentación técnica: Codenames por semilla, escalera de palabras (Hamming 1 + BFS), crucigrama (backtracking), adivina la palabra (entropía) y sopa de letras

1. Qué resuelve esta sección

La Sección 4 convierte el vocabulario de la plataforma en **cinco juegos de palabras**, cada uno con un algoritmo clásico como corazón: el **Codenames** (Indescifrable) reparte un tablero 5×5 con un generador congruencial lineal, de modo que una semilla de 6 caracteres reemplaza a un servidor; la **escalera de palabras** pide transformar una palabra alemana en otra cambiando una letra por paso — un camino en el grafo de Hamming 1 que se resuelve con BFS; el **crucigrama** coloca palabras alemanas entrelazadas con **backtracking**, con las pistas en español; **adivina la palabra** es el Wordle sobre el corpus, con el feedback de letras repetidas bien contado y la **entropía de Shannon** como medida de cada intento; y la **sopa de letras** esconde palabras alemanas que se buscan por su pista española. Todos comparten el principio de la plataforma: **sin APIs en vivo** — todo es determinista por semilla, así que compartir una partida es compartir un string.

Los juegos son además **pedagógicos**: las palabras salen del corpus de lecturas (Sección 1); cada peldaño de la escalera y cada intento del Wordle muestran su traducción, y las pistas del crucigrama y de la sopa son las glosas españolas del léxico. Jugar es repasar vocabulario.

2. Mapa de archivos

Pieza	Archivo	Qué contiene
Selección de palabras	pipeline/juegos.py	Extrae de lexico.json las formas ASCII (L=3–5) con glosa para la escalera y 400 lemas frecuentes no funcionales como entradas del crucigrama
Datos	src/data/juegos.json	escalera{L: {palabra: glosa}} y crucigrama[{palabra, pista}]; lo sobrescribe el pipeline
Motor escalera	src/engine/escalera.js (+ .test)	Grafo por cubetas comodín, BFS (camino mínimo y niveles), generación de retos por semilla, validación de pasos, estadísticas
Motor crucigrama	src/engine/crucigrama.js (+ .test)	Reglas de colocación, búsqueda por backtracking con presupuesto, numeración clásica, cuadrícula y métricas
Motor Wordle	src/engine/wordle.js (+ .test)	Feedback con letras repetidas (dos pasadas), secreto por semilla, filtro de candidatas consistentes y entropía de Shannon de un intento
Motor sopa	src/engine/sopa.js (+ .test)	Colocación aleatorizada con reintentos, relleno con la distribución de letras del pool, extracción/validación de selecciones en línea recta
Motor Codenames	src/engine/board.js (+ .test)	LCG por semilla, Fisher–Yates, composición/parseo de semilla con código de vocabulario

Pieza	Archivo	Qué contiene
Pools y disponibilidad	src/engine/juegos.js (+ .test)	Pool del corpus o de una lectura, criterios de disponibilidad por juego y selectores jugables (longitudes, pasos, tamaños)
UI	src/secciones/juegos/*.jsx + juegos.css	/juegos (lecturas + corpus), /juegos/:lectura (índice de juegos), /juegos/:lectura/:juego; Codenames en /juegos/codenames
Estadísticas	simulacion/juegos-stats.mjs	Ejecuta los motores reales → docs/datos-juegos.json (lo consume el PDF de métricas)

3. Datos: pipeline/juegos.py → juegos.json

El pipeline no inventa vocabulario: filtra el léxico de la Sección 1. Para la **escalera** toma las formas alemanas de 3–5 letras **solo ASCII** (sin umlauts ni ß: el alumno hispanohablante debe poder teclearlas) con su traducción. Para el **crucigrama** toma **lemas** de 4–9 letras atestiguados ≥ 2 veces en las lecturas (frecuencias de la Sección 2), excluyendo una lista curada de **palabras funcionales** (artículos, preposiciones, pronombres, auxiliares): «der» es frecuentísimo pero la pista «el / la» no enseña nada. Quedan las 400 más frecuentes, ordenadas por frecuencia.

```
{ "escalera": { "4": { "haus": "casa", "hand": "mano", ... }, ... },
  "crucigrama": [ { "palabra": "sagen", "pista": "decir" }, ... ] }
```

Los dos pools sirven a los cuatro juegos nuevos: **adivina la palabra** usa el diccionario de la escalera (mismas longitudes, mismas glosas) y la **sopa de letras** usa las entradas del crucigrama (palabra alemana + pista española). Además de los pools globales, el pipeline emite los MISMOS pools **por lectura** (partes de un libro agrupadas por título, como la Sección 3; el override léxico de la lectura manda sobre el léxico global, igual que en el Lector): el alumno puede jugar solo con el vocabulario de la lectura que está leyendo. Como el resto de datos pesados, juegos.json entra al frontend por **dynamic import**: chunk aparte, fuera del bundle inicial.

3.1 Disponibilidad por lectura

No toda lectura aguanta todo juego: una de principiante tiene ~30-40 palabras útiles y su grafo de Hamming apenas tiene aristas. En vez de una lista curada, src/engine/juegos.js aplica un **criterio formal por juego**: escalera si existe algún par a distancia ≥ 3 (hay reto que proponer); Wordle si alguna longitud reúne ≥ 12 palabras (con menos no hay incertidumbre que reducir); crucigrama y sopa si hay ≥ 6 entradas con pista. Los selectores de la UI salen de la misma fuente (pasosDisponibles, longitudesEscalera/Wordle, tamanosTablero), así que nunca ofrecen combinaciones vacías. La tabla medida por lectura está en metricas-seccion4.pdf §7.

4. Escalera de palabras: grafo de Hamming 1 + BFS

4.1 El modelo

Nodos = palabras del diccionario de longitud L; arista entre dos palabras si difieren en **exactamente una letra** (distancia de Hamming 1). Un reto es un par (origen, destino) y jugar es recorrer un camino: cada paso debe ser una palabra del diccionario vecina de la anterior. El «par» perfecto lo certifica el **camino mínimo**, que BFS encuentra en $O(V + E)$ por ser el grafo no ponderado.

4.2 Construcción por cubetas comodín

Comparar todas las parejas es $O(n^2 \cdot L)$. En su lugar, cada palabra se mete en L cubetas, una por posición enmascarada (hand → *and, h*nd, ha*d, han*): dos palabras son vecinas **si y solo si** comparten alguna cubeta. La construcción queda en $O(n \cdot L)$ más el tamaño real de la salida, y palabras de longitudes distintas nunca colisionan (la máscara conserva la longitud).

```
hand → { *and, h*nd, ha*d, han* }
land → { *and, l*nd, la*d, lan* }   comparten *and → vecinas
```

4.3 Generación de retos

generarReto(palabras, {pasos, semilla}) baraja los orígenes posibles con el LCG (determinista) y, para cada uno, calcula los niveles BFS y busca destinos a distancia **exacta** «pasos»: el reto anuncia su dificultad y la garantiza (no existe atajo más corto, porque la distancia ES el mínimo). La UI ofrece pista (siguiente palabra de un camino mínimo desde la posición actual, recalculado por BFS), deshacer y rendirse (muestra el camino del reto). Todo peldaño pisado enseña su glosa.

5. Crucigrama: colocación por backtracking

5.1 Reglas de colocación (legalidad)

El tablero es un mapa disperso celda → letra. Colocar una palabra en (fila, col, dir) es legal si: (1) la casilla anterior al inicio y la posterior al final están libres; (2) cada casilla de la palabra o está **vacía** o ya contiene **la misma letra** (eso es un cruce); y (3) toda casilla vacía que se rellena no toca lateralmente otra palabra (sin palabras paralelas pegadas). Además, salvo la primera palabra, toda colocación debe tener **al menos un cruce**: el tablero es conexo por construcción.

5.2 La búsqueda

Se parte de la palabra más larga del pool en horizontal (mejor ancla) y se profundiza: para cada palabra restante se generan sus colocaciones legales **ancladas en un cruce** (solo casillas donde ya está una de sus letras: el espacio de búsqueda se reduce a los anclajes posibles), ordenadas de más a menos cruces (tableros densos primero). Si una rama no alcanza el objetivo de n palabras, se **deshace** la colocación (se borran solo las casillas que esa palabra añadió, respetando los cruces) y se prueba la siguiente. Un **presupuesto de nodos** (5.000) acota el peor caso exponencial; si se agota, se reintenta con objetivo $n-1$ — con una palabra nunca falla, así que siempre devuelve tablero. En la práctica el presupuesto no se toca: la tasa de éxito medida es 100% (véase [metricas-seccion4.pdf](#)).

5.3 Salida

El resultado se traslada a origen (0, 0), se calculan las dimensiones y se numera al estilo clásico: casillas iniciales en orden de lectura; si una horizontal y una vertical empiezan en la misma casilla, comparten número. `cuadrícula()` lo vuelca a una matriz para la UI (`null` = casilla negra) y `metricas()` cuenta casillas, cruces y densidad.

6. Adivina la palabra: feedback exacto + entropía

6.1 El feedback con letras repetidas (dos pasadas)

La regla del Wordle parece trivial hasta que hay letras repetidas: cada letra del secreto puede justificar **una sola** marca. La implementación correcta (`evaluar`) hace dos pasadas: la primera casa los verdes y cuenta las letras del secreto que sobran; la segunda reparte amarillos solo mientras queden existencias de esa letra. Con una pasada, «anna» contra secreto «nase» marcaría dos amarillos para la «n» cuando el secreto solo tiene una.

```
secreto nase / intento anna → C C - - (una sola n justifica un solo amarillo)
```

6.2 El juego como teoría de la información

Cada intento induce una **partición** del conjunto de candidatas: una clase por patrón de feedback posible. La calidad del intento es la **entropía de Shannon** de esa partición — los bits que se esperan aprender: $H = -\sum p_i \cdot \log_2 p_i$ con $p_i = |\text{clase}_i| / |\text{candidatas}|$. `filtrarConsistentes` aplica la definición de consistencia (una candidata sobrevive si habría producido exactamente el mismo feedback en cada intento jugado — sin casos especiales para repetidas) y alimenta el contador «quedan N palabras posibles» de la UI, que hace visible la reducción del espacio; `entropiaDe/mejorIntento` alimentan el solver voraz de las métricas. Los intentos deben ser palabras del corpus (cada uno muestra su glosa: también equivocarse enseña vocabulario).

7. Sopa de letras: colocación aleatorizada

Es el hermano probabilista del crucigrama: como no hay restricción de conectividad, no hace falta backtracking — para cada palabra se sortean dirección y origen hasta encontrar una colocación compatible (celda vacía o misma letra; los solapes válidos crean cruces gratis) con un tope de sorteos por palabra. Dos decisiones con intención: solo **direcciones de lectura natural** ($\rightarrow$, $\downarrow$ y diagonal descendente, sin invertidas: es un juego para aprender vocabulario, no un test de visión) y el **relleno muestrea la distribución de letras de las palabras colocadas**, no la uniforme sobre a–z: las letras señuelo son plausibles en alemán y no delatan las palabras por contraste. La selección del jugador (click inicio $\rightarrow$ click fin) la valida buscarSeleccion: línea recta en una dirección válida, en el sentido de lectura o al revés, y con los extremos exactos de la palabra. Las pistas van en español: encontrar «regalo» es encontrar GESCHENK.

8. Codenames (Indescifrable): la semilla como servidor

El juego original necesita que dos capitanes vean la MISMA clave secreta sin comunicarse. Aquí lo resuelve la aritmética: la semilla (p. ej. DP-K9A2) codifica el vocabulario (DP = alemán principiante) y alimenta un **LCG** ($a = 1103515245$, $c = 12345$, $m = 2^{31}$) que baraja las palabras y reparte los colores con Fisher–Yates. Mismo string $\rightarrow$ mismo tablero y misma clave en cualquier dispositivo, sin backend. El barajado del resto de juegos reutiliza este LCG con una corrección (crearGeneradorNormalizado): el generador original puede emitir valores fuera de $[0, 1)$ con hashes negativos, y la parte fraccionaria lo normaliza sin alterar los tableros ya repartidos.

9. Motor puro (JS) y UI

Los cinco motores viven en src/engine/ sin DOM ni localStorage, testeados con Vitest (grafo y BFS sobre diccionarios de juguete y sobre el real; legalidad, determinismo e invariantes del crucigrama: cruces $\geq$ palabras – 1, sin conflictos de letra, numeración en orden de lectura; feedback del Wordle con repetidas; palabras de la sopa legibles en sus casillas; criterios de disponibilidad). La UI (src/secciones/juegos/) solo presenta y navega en DOS niveles, como la Sección 3: **/juegos** lista las lecturas (chip de nivel, nº de juegos disponibles) más «Todo el corpus» y el Codenames; **/juegos/:lectura** es el índice de juegos que ese vocabulario aguanta; **/juegos/:lectura/:juego** juega con ese pool. La **escalera** ofrece longitud y pasos (solo los jugables), input con validación y glosa por peldaño; el **crucigrama** renderiza la cuadrícula interactiva (avance de foco en la dirección activa, click repetido alterna horizontal/vertical, flechas y Backspace), comprobar y revelar; el **Wordle** muestra glosa y feedback por intento más el contador de candidatas consistentes; la **sopa** marca palabras con dos clicks (inicio y fin) y tacha la pista revelando la palabra alemana. El Codenames conserva su pantalla clásica en /juegos/codenames.

10. Decisiones y trampas

Tema	Decisión / trampa
Solo ASCII en escalera/crucigrama	Umlauts y ß se excluyen: en teclado español romperían la fluidez de tecleo. El grafo pierde ~5% de nodos (medido)
Distancia EXACTA en el reto	Buscar destino a distancia exacta k ($\text{no} \leq k$) hace que el «mínimo posible» anunciado sea verdad por construcción
Palabras funcionales fuera	Frecuencia alta $\neq$ buena entrada: «der/und/nicht» no enseñan nada como pista; lista curada en pipeline/juegos.py
Formas vs lemas	La escalera usa FORMAS (los peldaños solo deben existir); el crucigrama usa LEMAS (una entrada de crucigrama es una palabra de diccionario)
Presupuesto del backtracking	Garantiza terminación con peor caso exponencial; el reintento $n-1$ garantiza salida. Medido: nunca se activa con el pool real
Feedback del Wordle en dos pasadas	Con letras repetidas, una sola pasada marca amarillos de más; los verdes deben consumir primero (§6.1)
Sopa sin palabras invertidas	Solo $\rightarrow$, $\downarrow$ y diagonal descendente: leer al revés no enseña alemán; el relleno imita la distribución de letras del pool para no delatar por contraste

Tema	Decisión / trampa
Imports con extensión .js	Los motores se importan también desde node (simulacion/juegos-stats.mjs): los imports internos llevan extensión explícita

11. Cómo regenerar cada artefacto

```
python pipeline/juegos.py          # src/data/juegos.json (tras añadir lecturas)
npm test                          # tests de los motores
npm run simular-juegos            # docs/datos-juegos.json (estadísticas reales)
python docs/generar_doc_seccion4.py # este PDF
python docs/generar_metricas_seccion4.py
python docs/generar_autoaprendizaje_seccion4.py
```